



TED UNIVERSITY

Department of Computer Engineering

CMPE 491 – Senior Design Project I

TEDU Guide

High-Level Design Report

Deliverable D6 | Spring 2026

Team Members

1. Taci Furkan Gökalp
2. Elif Nazlı Özer
3. Fatima Nasir Muhammad

Advisor & Jury Members

Advisor: Tolga Kurtuluş Çapın

Jury Member 1: Mehmet Evren Çoşkun

Jury Member 2: Müslüm Bozyiğit

Submission Date: May 15, 2026

Table of Contents

1. Introduction	1
1.1 Purpose of the System	2
1.2 Design Goals	3
1.3 Definitions, Acronyms, Abbreviations	6
1.4 Overview	7
2. Current Software Architecture	5
3. Proposed Software Architecture	8
3.1 Overview	6
Figure 1: Application UI Mockup (Three-Screen Flow)	
Figure 2: Enhanced System Architecture Diagram	
3.2 Subsystem Decomposition	8
3.3 Hardware / Software Mapping	12
3.4 Persistent Data Management	
13	
Figure 3: Campus Database Entity-Relationship Diagram	
3.5 Access Control and Security	15
3.6 Global Software Control	16
Figure 4: Navigation Query Sequence Diagram	
Figure 5: Navigation Session State Machine	
3.7 Boundary Conditions	18
4. Subsystem Services	19
5. Glossary	22
6. References	23

1. Introduction

1.1 Purpose of the System

TEDU Guide is an AI-powered intelligent campus navigation system developed for TED University. Its primary purpose is to eliminate the persistent challenge of indoor wayfinding within TED University's multi-floor campus buildings by providing real-time, AI-assisted navigation through a mobile-compatible web application accessible on any modern browser without installation.

The system enables students, faculty, staff, and campus visitors to locate rooms, offices, laboratories, and facilities using natural language queries - both text and voice - and receive turn-by-turn navigation guidance on interactive floor plan maps. Indoor positioning is achieved through WiFi fingerprinting based on RSSI (Received Signal Strength Indicator) values, bypassing the GPS unavailability inherent to indoor environments.

1.2 Design Goals

The design of TEDU Guide is governed by the following prioritized design goals, derived from the Non-Functional Requirements (NFRs) established in the Analysis Report (D4):

Priority	Design Goal	Rationale / Constraint Source
1 Critical	Performance: ≤ 2 -second query response time	NFR-01: user experience, real-time navigation
2 Critical	Privacy by Design: KVKK compliance, session-only location data	NFR-06, ethical constraint; Turkish Law No. 6698
3 High	Scalability: ≥ 100 concurrent users without degradation	NFR-02: campus peak hours (class breaks)
4 High	Maintainability: modular, independently updatable subsystems	NFR-07: sustainability constraint; future teams
5 High	Accessibility: voice-only mode; step-free routing	NFR-04, FR-05, FR-09: social and ethical constraints

6 Medium	Robustness: graceful degradation (offline 30–90s, NLP fallback)	FR-10: real-world network reliability
Priority	Design Goal	Rationale / Constraint Source
7 Medium	Portability: cloud-deployable, no proprietary infrastructure	NFR-08: zero-budget constraint

1.3 Definitions, Acronyms, Abbreviations

Term	Definition
RSSI	Received Signal Strength Indicator -radio signal power measurement (dBm) from a WiFi access point.
WiFi Fingerprinting	Indoor positioning technique matching observed RSSI vectors against a precollected reference database.
k-NN	k-Nearest Neighbors - the supervised ML algorithm used for location estimation.
NLP	Natural Language Processing -AI-based human language understanding.
LLM	Large Language Model - GPT-4 (OpenAI) used for intent recognition and query parsing.
REST API	Representational State Transfer Application Programming Interface -stateless HTTP-based service interface.
PostGIS	PostgreSQL extension for storing, indexing, and querying geographic/spatial data.

JWT	JSON Web Token - compact, signed token used for admin authentication.
HTTPS/TLS	HyperText Transfer Protocol Secure / Transport Layer Security - encrypted communication protocol.
KVKK	Kişisel Verilerin Korunması Kanunu - Turkish Law No. 6698 on Protection of Personal Data.
CDN	Content Delivery Network - geographically distributed servers for fast static asset delivery.
PWA	Progressive Web Application - web app with offline-capable caching via Service Worker.
AP	Access Point - WiFi hardware node in the campus network infrastructure.
A*	A-Star - a graph pathfinding algorithm used for optimal route computation.

1.4 Overview

This report is structured as follows: Section 2 acknowledges the absence of an existing software architecture at TED University for indoor navigation. Section 3 presents the full proposed software architecture including subsystem decomposition, hardware/software mapping, data management strategy, access control design, global control flow, and boundary condition handling. Section 4 details the service interfaces of each subsystem. Sections 5 and 6 provide a glossary and reference list. Five rich visual figures (UI mockup, architecture, database ERD, sequence diagram, and state machine) are embedded throughout Section 3 to complement the textual descriptions.

2. Current Software Architecture

TED University currently operates no software system for indoor campus navigation. The university relies entirely on static physical signage, printed floor plans available in building lobbies and on the intranet, and manual human assistance. As documented in the Analysis Report (D4,

Section 2), these mechanisms present four core limitations: no real-time positioning, no dynamic route guidance, static non-updatable information, and no accessibility support.

There is therefore no legacy software architecture to integrate with, migrate from, or constrain the TEDU Guide design. The proposed architecture is a greenfield implementation. However, the following existing infrastructure assets are treated as given constraints:

- **Campus WiFi Network:** Existing 802.11 access point infrastructure across campus buildings - the hardware base for RSSI fingerprinting (Economic Constraint: no new hardware).
- **University IT Systems:** Student information and directory data (office assignments, faculty names) - a potential future integration for database population, subject to IT policy approval.
- **University Internet:** Static floor plan images available as a baseline for digitization into interactive Leaflet.js maps.

Design implication: Because there is no legacy software to maintain backward compatibility with, subsystem boundaries can be drawn optimally for the TEDU Guide requirements without compromise.

3. Proposed Software Architecture

3.1 Overview

TEDU Guide is organized into a **three-layer client-server architecture** with an external AI services integration tier. All inter-layer communication is encrypted via HTTPS/TLS. The three primary layers are:

1. **Client Layer** - The browser-based React.js frontend, responsible for rendering the UI, collecting WiFi RSSI signals via the browser Web API, and presenting navigation results.
2. **Server Layer** - The Python FastAPI backend, responsible for query routing, NLP integration, indoor positioning inference, route computation, and AI output validation.
3. **Data Layer** - The PostgreSQL/PostGIS campus spatial database and the local k-NN RSSI fingerprint model, storing all persistent campus data.

An external **AI Services tier** (OpenAI GPT-4 API) is accessed exclusively by the Server Layer and is never directly exposed to the client.

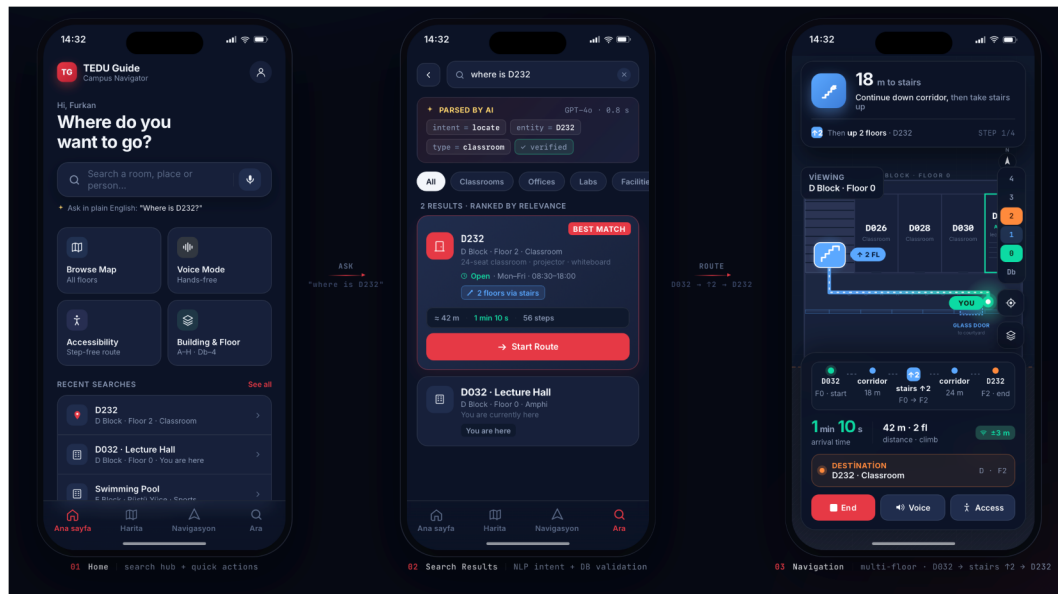


Figure 2: TEDU Guide — Proposed System Architecture

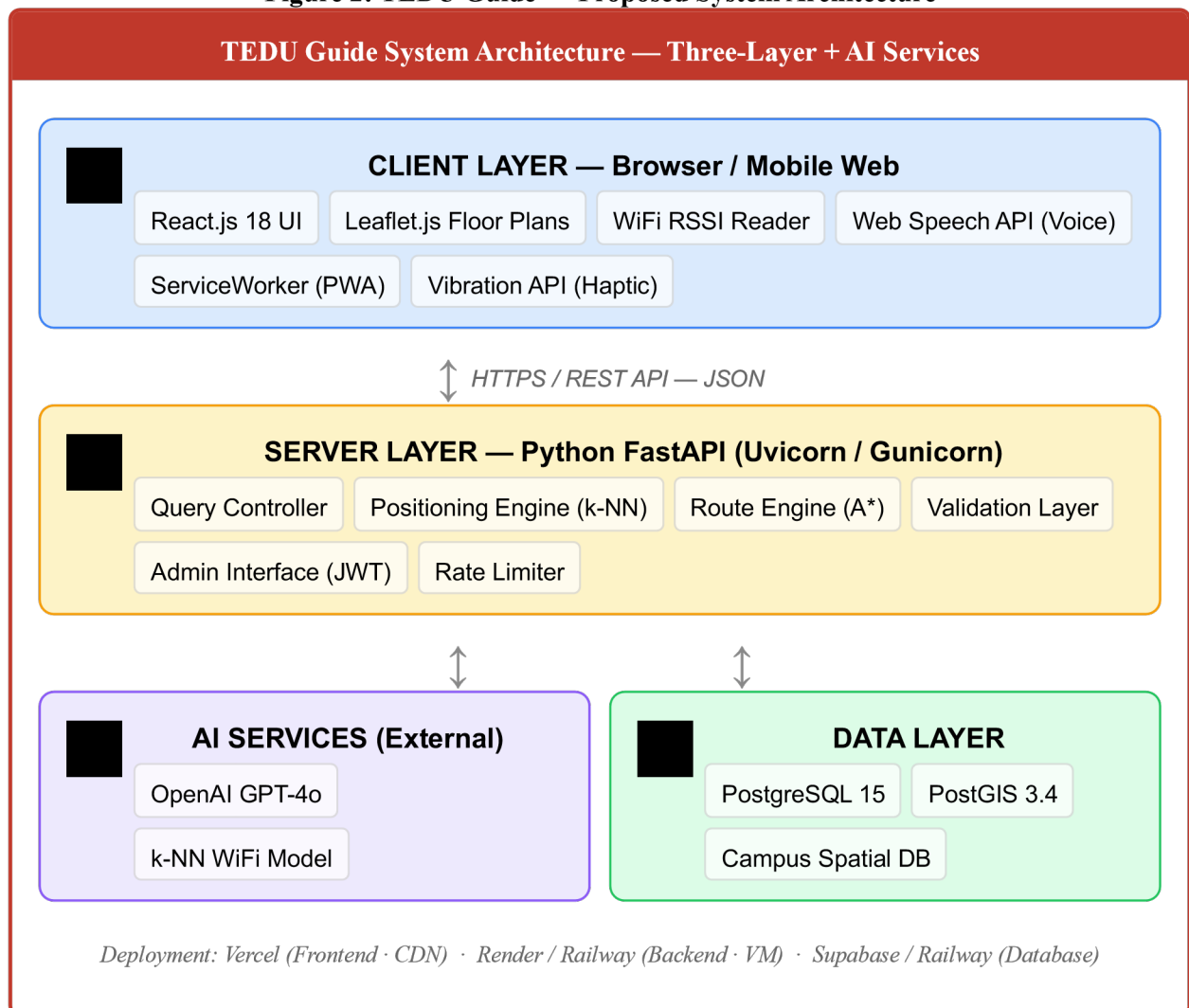


Figure 2: Three-layer + AI services architecture. Client layer runs entirely in the browser (no install required). Server layer is stateless (JWT auth, async FastAPI). AI services are external integrations accessed only by the server. Data layer uses PostGIS for spatial queries. All traffic encrypted HTTPS/TLS.

3.2 Subsystem Decomposition

The TEDU Guide system is decomposed into seven primary subsystems. Each subsystem has a clearly defined responsibility boundary, a documented interface, and can be updated or replaced independently (satisfying NFR-07 -Maintainability).

SS-01: Frontend UI Subsystem

Technology: React.js (v18+), Leaflet.js (v1.9+), Web Speech API, Vibration API

Responsibility: Renders all user-facing screens (Home, Search, Map View, Navigation Mode, Arrived). Manages client-side application state via React Context/Hooks. Hosts the WiFi RSSI collection logic using the browser's Network Information API.

Key Components:

- *SearchBar* — text and voice query entry with autocomplete
- *MapCanvas* — Leaflet.js floor plan renderer with route overlay
- *NavigationPanel* — turn-by-turn instruction display with directional arrow
- *AccessibilityToggle* — enables step-free routing mode
- *RSSICollector* — polls visible APs and sends vectors to backend
- *ServiceWorker* — caches last floor plan + route for 30–90s offline (FR-10)

Interfaces provided: User interface (browser). **Interfaces required:** REST API of SS-02.

SS-02: API Gateway Subsystem

Technology: Python 3.11+, FastAPI, Uvicorn/Gunicorn, Pydantic

Responsibility: Serves as the single entry point for all client requests. Routes queries to the appropriate downstream subsystem (NLP, Positioning, Route Engine). Enforces HTTPS, validates request schemas, applies rate limiting, and returns structured JSON responses.

Key Routes:

- POST /api/query — receives NLP query + optional RSSI vector
- GET /api/route/{from_id}/{to_id} — route computation
- GET /api/location/search?q= — campus DB text search
- GET /api/floors/{building_id} — floor plan GeoJSON
- POST /api/admin/* — authenticated CRUD operations

Interfaces provided: REST API (HTTPS/JSON). **Interfaces required:** SS-03, SS-04, SS-05, SS-07.

Technology: OpenAI Python SDK (GPT-4o / gpt-4-turbo), structured output (JSON mode)

Responsibility: Parses free-form user queries (text or voice-transcribed) to extract a structured *intent* (navigate | locate | query_hours | general_info) and named *entities* (room number, person name, facility type, floor). Returns a structured JSON object to the API Gateway.

Design Decision - Validation Layer: All NLP-extracted entities are cross-checked against the Campus Database (SS-07) before being presented to the user, preventing hallucinated room numbers or names from reaching the user (Algorithmic Accountability -D2, §1.3).

Fallback Behavior: If GPT-4 returns ambiguous or unverifiable output, the system returns a clarification prompt to the user rather than presenting incorrect information.

Technology: Python scikit-learn (KNeighborsClassifier/Regressor), NumPy, joblib

Responsibility: Maintains the trained k-NN model against the RSSI fingerprint database. At runtime, receives a vector of {BSSID: RSSI_dBm} pairs from the client, applies the k-NN inference, and returns an estimated coordinate (building, floor, x, y) with a confidence score.

Recalibration Workflow: An administrator can trigger model retraining via the Admin Interface (SS-06) by uploading a new fingerprint dataset CSV. The model is retrained and hot-swapped without system restart (satisfying Sustainability Constraint).

Accuracy bound: 3–5 meters under standard conditions (FR-02 / NFR-02).

SS-05: Route Engine Subsystem

Technology: Python NetworkX library (graph representation), A* algorithm implementation

Responsibility: Maintains an in-memory weighted graph of the campus floor plan where nodes represent navigable waypoints (corridor intersections, room entrances, stairwell landings, elevator stops) and edges represent walkable segments with weights proportional to distance. Computes the shortest path between any two nodes via A*.

Accessibility Mode: When the accessibility flag is set (FR-05), all edges passing through stairs or steps are removed from the graph, restricting paths to elevator-accessible routes only.

Multi-floor routing: Floor transitions are modeled as special edges (type: staircase | elevator) connecting floor graph layers.

Technology: React.js frontend panel, FastAPI backend endpoints, JWT authentication

Responsibility: Provides authorized university staff with a secured web interface to manage the campus database: add/edit/remove location entries, update room assignments, modify facility operating hours, and trigger WiFi fingerprint model recalibration. All actions are logged with timestamp and administrator ID.

Authentication: Admin login via university credentials with JWT session tokens (15-minute expiry, refresh token pattern). No admin routes are accessible without a valid JWT.

SS-07: Data Persistence Subsystem

Technology: PostgreSQL 15+, PostGIS 3.3+, hosted on Supabase / Railway

Responsibility: Stores all persistent campus data: spatial location records, floor plan graph nodes/edges, faculty-office assignments, facility operating hours, and the RSSI fingerprint reference dataset. Provides spatial query capabilities via PostGIS (proximity search, coordinate lookup).

Session data: User location data is held in server-side memory for the duration of the active navigation session only - never written to the database without explicit user opt-in (KVKK / NFR- 06).

3.3 Hardware / Software Mapping

The following table maps each subsystem to its deployment target and the underlying hardware/software infrastructure. All deployment targets use free or free-tier cloud hosting to satisfy the Economic Constraint (zero project budget).

Subsystem	Deployment Target	Hardware / Platform	Rationale
SS-01 Frontend UI	Vercel (CDN edge network)	Cloud server + global CDN	Free tier, global edge delivery, auto-deploy from GitHub
SS-02 API Gateway	Render / Railway (Python)	Cloud VM (512 MB RAM)	Free tier, Docker support, FastAPI compatible
SS-03 NLP / AI	External: OpenAI API	OpenAI cloud infrastructure	Pay-per-use; GPT-4o for production

SS-04 Positioning	Co-located with SS-02	Same VM as API Gateway	Avoids network latency for ML inference
SS-05 Route Engine	Co-located with SS-02	Same VM as API Gateway	In-memory graph, no separate process needed

SS-06 Admin Interface	Vercel (UI) + Render (API)	Same as SS-01 / SS-02	Re-uses existing deployment pipelines
SS-07 Data Persistence	Supabase / Railway PostgreSQL	Cloud managed PostgreSQL	Free tier, PostGIS available, managed backups
WiFi AP TED University Existing hardware (Economic Constraint)	802.11	No new hardware procurement	Network campus buildings

Network boundary: Client devices communicate only with the Vercel CDN (static assets) and the Render/Railway API server (dynamic requests). The API server communicates outbound to OpenAI API and to the Supabase database. Client devices never directly access the database or the AI service.

3.4 Persistent Data Management

The Data Persistence Subsystem (SS-07) uses PostgreSQL with the PostGIS extension. The schema is designed to be extensible with minimal structural changes when new buildings or room assignments are added (Sustainability Constraint). The following core tables are defined:

```
-- Campus Location Entities CREATE
TABLE buildings (
  id          SERIAL PRIMARY KEY,      name
  VARCHAR(100) NOT NULL,
  code        VARCHAR(20) UNIQUE NOT NULL, -- e.g. "BLOCK_A"      geom
  GEOMETRY(Polygon, 4326) );

CREATE TABLE floors (
  id          SERIAL PRIMARY KEY,      building_id INT REFERENCES
  buildings(id),      floor_number INT NOT NULL,      svg_map_url TEXT,
  -- Leaflet overlay SVG      UNIQUE(building_id, floor_number)
);
```

```

CREATE TABLE locations (
    id SERIAL PRIMARY KEY, floor_id INT REFERENCES
floors(id), room_number VARCHAR(20), name VARCHAR(200),
-- "Prof. Çapın's Office" type VARCHAR(50), --
classroom|office|lab|facility coordinates GEOMETRY(Point, 4326),
operating_hours JSONB, -- {"mon":"09:00-17:00", ...}
is_accessible BOOLEAN DEFAULT TRUE, updated_at TIMESTAMP DEFAULT NOW()
);

-- Navigation Graph CREATE TABLE nav_nodes ( id SERIAL PRIMARY KEY,
floor_id INT REFERENCES floors(id), node_type VARCHAR(30),
- corridor|room_entrance|staircase geom GEOMETRY(Point, 4326),
is_accessible BOOLEAN DEFAULT TRUE );

CREATE TABLE nav_edges ( id SERIAL PRIMARY KEY, from_node
INT REFERENCES nav_nodes(id), to_node INT
REFERENCES nav_nodes(id), weight FLOAT, --
distance in meters edge_type VARCHAR(20), --
walk|staircase|elevator is_accessible BOOLEAN DEFAULT TRUE );

-- WiFi Fingerprint Database CREATE TABLE
rssi_fingerprints ( id SERIAL PRIMARY
KEY, location_id INT REFERENCES locations(id),
ap_bssid VARCHAR(20), rssi_mean FLOAT,
rssi_std FLOAT, recorded_at TIMESTAMP DEFAULT
NOW() );

-- Admin Audit Log CREATE TABLE
admin_actions ( id SERIAL
PRIMARY KEY, admin_id INT,
action VARCHAR(50),
table_name VARCHAR(50),

```

```

record_id INT, performed_at
TIMESTAMP DEFAULT NOW() );

```

Figure 3: Campus Database -Entity-Relationship Diagram

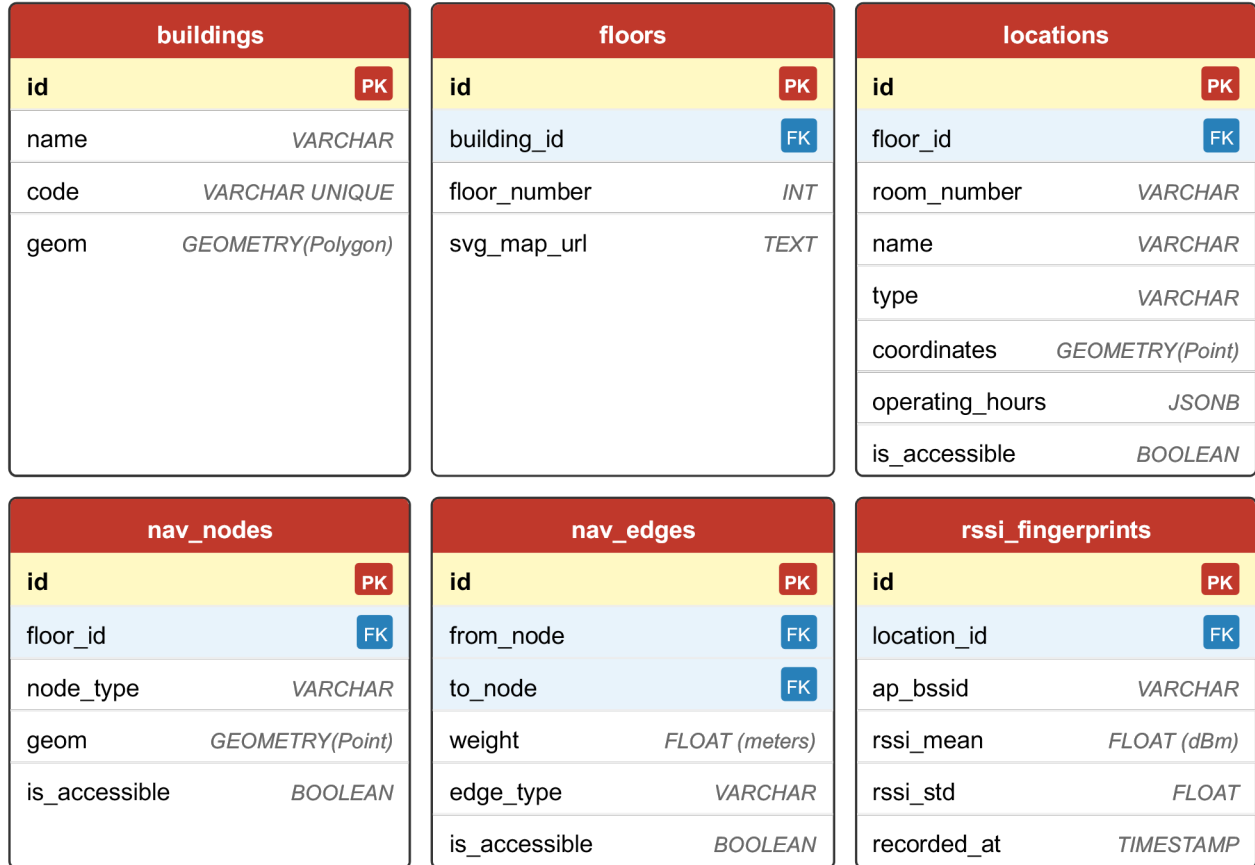


Figure 3: Campus database ERD. Yellow rows = Primary Keys; blue rows = Foreign Keys. The buildings → floors → locations hierarchy forms the spatial campus model. nav_nodes / nav_edges form the A* routing graph. rssi_fingerprints is the WiFi k-NN training dataset. All geometry columns use PostGIS with EPSG:4326 CRS.

Session data handling: User location data (estimated coordinates from k-NN) is stored exclusively in the FastAPI server's in-memory session store (Python dictionary keyed by session UUID) for the duration of the active navigation session. It is never written to the PostgreSQL database. Upon session termination, the session entry is deleted from memory (KVKK compliance, NFR-06).

3.5 Access Control and Security

TEDU Guide distinguishes three access levels: *Guest/User* (unauthenticated), *Authenticated User* (future extension), and *Administrator*.

Role	Authentication	Permissions
------	----------------	-------------

Guest / Campus User	None required (FR-12)	Navigate campus, search locations, view floor plans, use voice mode, use accessibility mode. Cannot modify any data.
Administrator	Username + password → JWT (15 min expiry + refresh token)	All Guest permissions + full CRUD on campus database, location records, facility hours. Trigger fingerprint model recalibration. View admin audit log.

Transport security: All client–server communication is enforced over HTTPS/TLS. The API server rejects any non-TLS connection.

API security: Admin endpoints are protected by JWT Bearer token validation in FastAPI middleware. Invalid or expired tokens return HTTP 401. All admin actions are recorded in the audit log.

Input validation: All API request bodies are validated via Pydantic schemas before processing. SQL injection is prevented by using SQLAlchemy ORM with parameterized queries exclusively.

Data privacy: OpenAI API calls contain only the user's text query - no PII or location coordinates are sent to OpenAI. This satisfies KVKK data minimization.

3.6 Global Software Control

The system follows a **request-response control flow** on the server side (stateless REST) and an **event-driven reactive model** on the client side (React hooks/context).

3.6.1 Client-Side Control Flow

The React frontend uses a centralized `NavigationContext` (React Context API) to hold the global application state: current session, query results, active route, user position estimate, and accessibility mode flag. State transitions are triggered by user events (query submit, navigation start, destination arrival) and API responses. The ServiceWorker intercepts fetch requests to serve cached responses during network disconnections.

3.6.2 Server-Side Control Flow

The FastAPI backend operates as a stateless REST service. Long-running operations (OpenAI API call) are executed asynchronously using Python's `asyncio` + FastAPI's async endpoint support.

The overall query processing pipeline for a navigation request:

1. Receive POST `/api/query` with `{text, rssi_vector, accessibility_mode}`

2. Validate request schema (Pydantic)
3. Async: send text to SS-03 (OpenAI GPT-4) → receive {intent, entities}
4. Parallel: send rssi_vector to SS-04 (k-NN) → receive {estimated_position}
5. Validate entities against SS-07 (Campus DB) - reject hallucinated results
6. Send {from: estimated_position, to: validated_entity, accessibility: flag} to SS-05 (Route Engine)
7. Return {route_waypoints, turn_instructions, map_overlay_geojson} to client

Figure 4: Navigation Query - UML Sequence Diagram

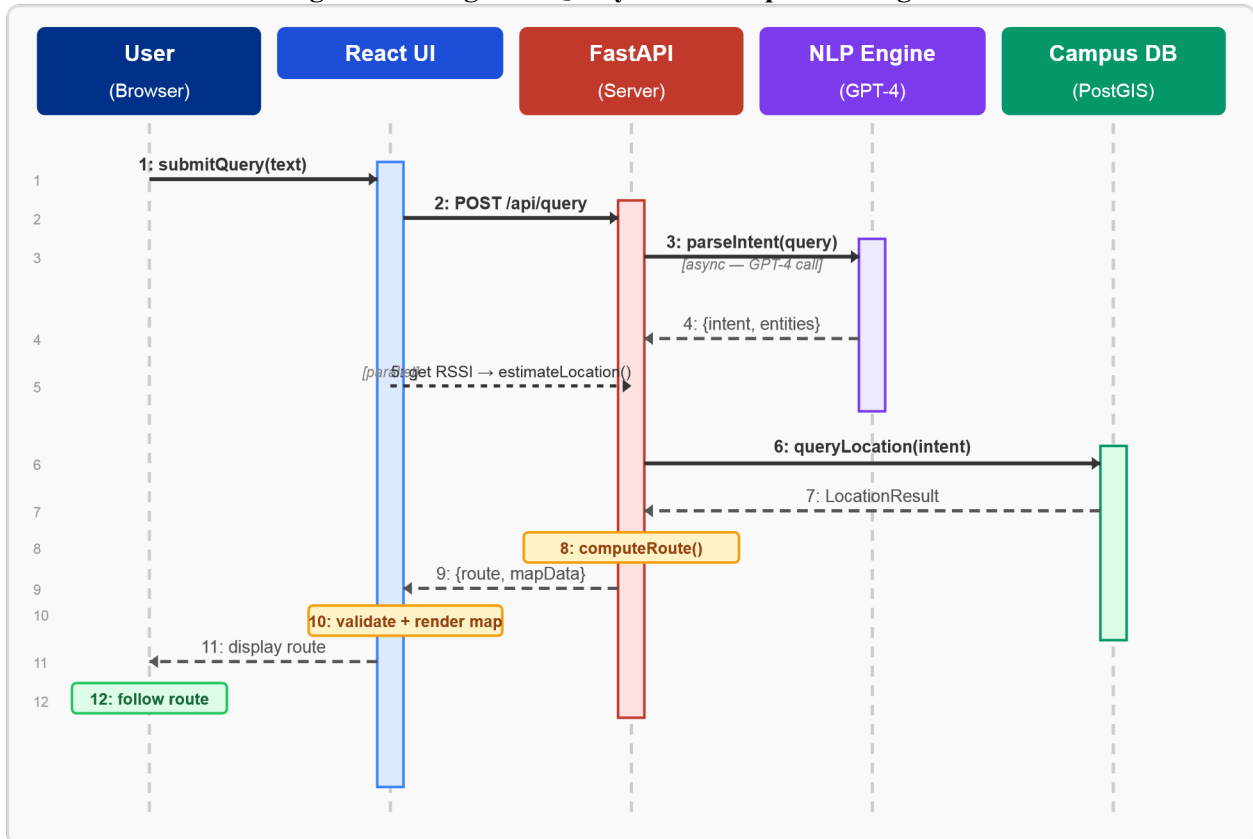


Figure 4: Navigation query sequence diagram. Steps 3 (NLP) and 5 (RSSI) execute asynchronously/in parallel to minimize latency. Step 8 (route computation) occurs server-side before result is returned. Step 10 (validation + render) cross-checks AI output against campus DB before displaying to user. Yellow boxes indicate internal server processing steps; dashed arrows indicate return messages.

3.6.3 Concurrency Model

The server is deployed with **Gunicorn + Uvicorn workers** (4 async workers), supporting concurrent handling of up to 100 simultaneous users (NFR-02) on the free-tier cloud VM. Database connection pooling (SQLAlchemy's AsyncSession with pool_size=10) prevents connection exhaustion under peak load.

3.7 Boundary Conditions

Figure 5: Navigation Session - State Machine Diagram

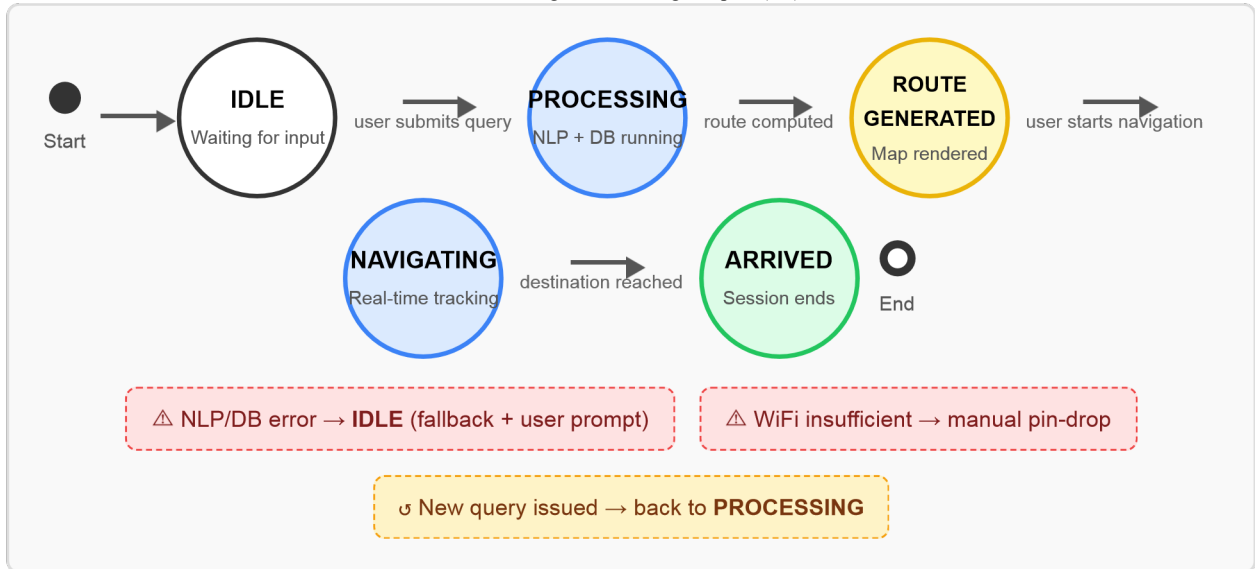


Figure 5: Navigation session lifecycle. The session begins in **IDLE** and progresses through **PROCESSING** (NLP + positioning), **ROUTE GENERATED** (map rendered for review), and **NAVIGATING** (real-time tracking) to **ARRIVED** (terminal state). Error transitions (dashed red) trigger graceful fallback behavior. The "new query issued" back-edge allows mid-journey re-routing without session termination.

3.7.1 System Startup

On application startup, the FastAPI server performs: (1) loads the pre-trained k-NN model from the serialized joblib file; (2) initializes the campus navigation graph from PostgreSQL into memory; (3) establishes the database connection pool; (4) verifies the OpenAI API key environment variable. If any critical step fails, the server logs the error and exits with a non-zero status code, triggering an auto-restart.

3.7.2 System Shutdown

On receiving SIGTERM (graceful shutdown), the server completes in-flight requests (30-second drain window), flushes the audit log, closes database connections, and clears all in-memory session data. No user data persists after shutdown.

3.7.3 Network Disconnection

If the client loses network connectivity during an active navigation session, the ServiceWorker serves the cached floor plan and last-computed route from the browser cache. The system remains functional for 30–90 seconds of disconnection (FR-10). The WiFi positioning status indicator notifies the user of the offline state.

3.7.4 NLP / OpenAI API FaOlure

If the OpenAI API returns an error (rate limit, timeout, or service unavailability), the server falls back to a deterministic keyword-matching search against the campus database. The user is informed that AI-enhanced query processing is temporarily unavailable but basic location search remains functional.

3.7.5 Insufficient WiFi Signal

If the RSSI vector contains fewer than 3 visible access points (insufficient for reliable k-NN inference), the positioning engine returns a low-confidence flag. The client prompts the user to manually select their starting location from the interactive floor plan (pin-drop fallback).

4. Subsystem Services

This section formally specifies the interface exposed by each subsystem.

SS-01: Frontend UI - Services Consumed

Service	Provider	Interface
Query processing	SS-02 API Gateway	POST /api/query (JSON)
Location search	SS-02 API Gateway	GET /api/location/search?q=
Route retrieval	SS-02 API Gateway	GET /api/route/{from}/{to}
Floor plan GeoJSON	SS-02 API Gateway	GET /api/floors/{building_id}
Voice transcription	Browser Web Speech API	SpeechRecognition (browser native)
Haptic feedback	Browser Vibration API	navigator.vibrate() (browser native)

SS-02: API Gateway -Services Provided

Endpoint	Method	Request Body / Params	Response
/api/query	POST	{text: str, rssi: {bssid:rssi}, accessible: bool}	{intent, entities, route, instructions}
/api/location/search	GET	?q=string&type=filter	[{id, name, room, building, floor, type}]
/api/route/{f}/{t}	GET	from_id, to_id, accessible (query param)	{waypoints[], instructions[], geojson}

/api/floors/{bid}	GET	building_id	{floors: [{id, number, svg_url, geojson}]}
/api/admin/locations	GET/POST/PUT/DELETE	JWT Bearer + location record JSON	Location record(s)
Endpoint	Method	Request Body / Params	Response
/api/admin/recalibrate	POST	JWT Bearer + fingerprint CSV upload	{status, accuracy_estimate}

SS-03: NLP Subsystem - Internal Interface

Function	Input	Output	Error Behavior
parse_intent(query: str) → Intent	Raw user text	{intent: navigate locate query_hours general, entities: {name?, room?, type?, floor?}}	Returns {intent: unknown, fallback_query: str}

SS-04: Positioning Subsystem - Internal Interface

Function	Input	Output	Error Behavior
estimate_location(rssi_vec: dict) → Position	{bssid: rssi_dBm, ...}	{building, floor, x, y, confidence: 0–1}	Returns {confidence: 0, fallback: true} if <3 APs visible

retrain_model(csv_path: str) → ModelMetrics	Path to new fingerprint CSV	{accuracy_m, k_value, timestamp}	Rolls back to previous model on failure
---	-----------------------------	----------------------------------	---

SS-05: Route Engine - Internal Interface

Function	Input	Output	Error Behavior
compute_route(from: Position, to: LocationID, accessible: bool) → Route	Start position, destination ID, accessibility flag	{waypoints[], instructions[], total_distance_m, floors_involved[]}	Returns {error: "no_path"} if destination unreachable

SS-07: Data Persistence - Service Interface

Operation	ORM Method	Description
Location lookup by name	search_by_name(q: str)	Full-text + fuzzy search on name and room_number fields
Nearest locations	find_nearest(point: Geometry, radius_m: float)	PostGIS ST_DWithin spatial proximity query
Graph load	load_nav_graph(building_id: int)	Loads all nav_nodes and nav_edges into NetworkX graph
Location CRUD	create / update / delete location	Admin-only; logs to admin_actions table

5. Glossary

Term	Definition
RSSI	Received Signal Strength Indicator - power level of a received WiFi signal, expressed in dBm (decibels relative to 1 milliwatt).
WiFi Fingerprinting	Indoor positioning technique: a site-survey phase records RSSI vectors at known reference points; at runtime, the observed vector is matched against the database to estimate location.
k-NN	k-Nearest Neighbors - supervised ML classification/regression algorithm that estimates a location by finding k stored fingerprints most similar to the current observation.
NLP / LLM	Natural Language Processing / Large Language Model. In TEDU Guide, GPT-4 (OpenAI) serves as the NLP engine for intent recognition and entity extraction from free-form queries.
A* (A-Star)	Informed graph search algorithm used by the Route Engine. Uses a heuristic (Euclidean distance to goal) to efficiently find the shortest path in the navigation graph.
REST API	Representational State Transfer API - stateless HTTP-based interface where each request contains all information needed to process it independently.
PostGIS	Spatial extension for PostgreSQL enabling storage, indexing, and querying of geographic objects and coordinates.
GeoJSON	Open standard format for encoding geographic data structures (points, lines, polygons) as JSON, used for floor plan and route data exchange.

JWT	JSON Web Token - compact, cryptographically signed token used to authenticate admin sessions without server-side session storage.
KVKK	Kişisel Verilerin Korunması Kanunu - Turkish personal data protection law (Law No. 6698) governing collection, processing, and storage of personal data.
ServiceWorker	Browser-native background script that intercepts network requests and serves cached content, enabling short-duration offline functionality (PWA pattern).
Subsystem	A coherent group of software components with a defined responsibility boundary and interface, independently deployable and maintainable.

Term	Definition
Validation Layer	Backend component that cross-checks AI-generated entities against the structured campus database to prevent hallucinated or incorrect information from reaching users.
Accessibility Mode	Navigation mode restricting route computation to step-free, elevator-accessible paths for users with mobility impairments.

6. References

1. ACM. (2018). *ACM Code of Ethics and Professional Conduct*. Association for Computing Machinery. <https://www.acm.org/code-of-ethics>
2. Aggarwal, P., & Chugh, N. (2022). Review of machine learning techniques for indoor positioning. *Journal of Location Based Services*, 16(4), 280–322. <https://doi.org/10.1080/17489725.2022.2161198>
3. Bahl, P., & Padmanabhan, V. N. (2000). RADAR: An in-building RF-based user location and tracking system. *Proceedings of IEEE INFOCOM 2000*, 2, 775–784. <https://doi.org/10.1109/INFCOM.2000.832252>
4. Brusilovsky, P., & Millán, E. (2007). User models for adaptive hypermedia and adaptive educational systems. In *The Adaptive Web* (pp. 3–53). Springer. https://doi.org/10.1007/978-3-540-72079-9_1
5. Bynum, T. W. (2023). *Computer and Information Ethics*. Stanford Encyclopedia of Philosophy. <https://plato.stanford.edu/entries/ethics-computer/>
6. FastAPI. (2024). *FastAPI Documentation*. Tiangolo. <https://fastapi.tiangolo.com>
7. IEEE. (n.d.). *IEEE Code of Ethics*. Institute of Electrical and Electronics Engineers. <https://www.ieee.org/about/corporate/governance/p7-8.html>
8. IEEE Computer Society. (1999). *IEEE Software Engineering Code of Ethics and Professional Practice (Version 5.2)*. IEEE/ACM. <https://www.computer.org/education/code-of-ethics>
9. Liu, H., Darabi, H., Banerjee, P., & Liu, J. (2007). Survey of wireless indoor positioning techniques and systems. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, 37(6), 1067–1080. <https://doi.org/10.1109/TSMCC.2007.905750>
10. OpenAI. (2024). *GPT-4 Technical Report*. OpenAI. <https://openai.com/research/gpt-4>
11. PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/15/>
12. React Team. (2024). *React Documentation*. Meta Open Source. <https://react.dev>
13. Republic of Turkey. (2016). *Kişisel Verilerin Korunması Kanunu - Law No. 6698*. Official Gazette No. 29677.
14. Vladimir Agafonkin. (2024). *Leaflet.js Documentation*. <https://leafletjs.com/reference.html>

TED UNIVERSITY - Department of Computer Engineering

CMPE 491 Senior Design Project I - Spring 2026

TEDU Guide High-Level Design Report (D6) - Submitted: May 15, 2026